

Praxis: Navigation im Kernel- Quellcode

Symbole, Treiber, Device Trees und Build-Pfade systematisch erschließen

1.10

Quellcode-Navigation beginnt mit einer klaren Frage

01 Hardwarebezug bestimmen

Welche Geräteerkennung, welches Device-Tree-compatible oder welches Fehlersymptom ist bekannt?

02 Passendes Subsystem eingrenzen

GPIO, I²C, SPI, Netzwerk, Speicher oder Architektur vor der globalen Suche auswählen.

03 Implementierung und Build verbinden

Treiberdatei, öffentliche Struktur, Probe-Funktion, Kconfig und Makefile gemeinsam lesen.

04 Ergebnis überprüfbar festhalten

Dateipfad, Symbol, Quellstand und nachvollziehbare Suche für die spätere Analyse notieren.

Leitfrage dieser Übung: Wie wird der GPIO-Controller snps,dw-apb-gpio im Kernel aktiviert?

Wichtige Verzeichnisse geben der Suche Richtung

[drivers/gpio/](#)

GPIO-Controller-Treiber und Subsysteminfrastruktur

[include/linux/gpio/](#)

Interne GPIO-Header und zentrale Datenstrukturen

[arch/arm64/boot/dts/](#)

Board- und SoC-spezifische Device-Tree-Dateien

Documentation/

Bindings, Subsystemdokumentation und Schnittstellen

Kconfig und Makefile im jeweiligen Subsystem verbinden Quellcode, Optionen und Build-Auswahl.

git grep findet Symbole direkt im Repository

GETRACKTE QUELLDATEIEN DURCHSUCHEN

```
$ cd linux-src
$ git rev-parse --short HEAD

$ git grep -n 'dwapb_gpio_probe'
$ git grep -n 'CONFIG_GPIO_DWAPB'
$ git grep -n -F 'snps,dw-apb-gpio'
```

01 Zeilennummern

-n ergänzt den Treffer um eine direkt nachvollziehbare Dateiposition.

02 Literal statt Regex

-F behandelt das Suchmuster als festen Text und vermeidet Regex-Überraschungen.

Der aktuelle Git-Commit macht Suchergebnisse, Codebezüge und spätere Vergleiche reproduzierbar.

Suchbereich, Kontext und Muster gezielt eingrenzen

PATHSPEC · KONTEXT · REGULÄRE AUSDRÜCKE

```
$ git grep -n -F 'snps,dw-apb-gpio' -- drivers/gpio/ arch/arm64/boot/dts/ \
Documentation/devicetree/bindings/
$ git grep -n -C 3 'GPIO_DWAPB' -- drivers/gpio/Kconfig drivers/gpio/Makefile
$ git grep -n -E 'dwapb_.*(probe|irq)' -- drivers/gpio/gpio-dwapb.c
```

-- trennt Suchmuster und Pfade; -C liefert Kontext, -E ermöglicht erweiterte reguläre Ausdrücke.

Definition, Deklaration und Aufrufer unterscheiden

SYMBOLBASIERTE SUCHE

```
$ git grep -n 'struct gpio_chip' -- \
    include/linux/gpio/ drivers/gpio/

$ git grep -n 'dwapb_gpio_add_port' -- \
    drivers/gpio/gpio-dwapb.c

$ git grep -n 'devm_gpiocchip_add_data' -- \
    drivers/gpio/ include/linux/gpio/
```

01 Struktur verstehen

Header beschreibt den Vertrag; konkrete Treiber enthalten Instanzen und Nutzung.

02 Aufrufweg verfolgen

Eine Funktion kann definiert, referenziert oder indirekt über Callbacks verwendet werden.

Ein Texttreffer ist noch kein Aufrufbeweis: Definition, Referenz und Kontrollfluss getrennt prüfen.

Interne Header erklären den Vertrag eines Treibers

SCHEMATISCH · KEIN VOLLSTÄNDIGER HEADER

```
#include <linux/gpio/driver.h>

struct gpio_chip {
    /* Richtung, Werte, IRQ und Metadaten */
    /* konkrete Felder: Header + Doku prüfen */
};

devm_gpiochip_add_data(dev, chip, data);
```

01 API zuerst lesen

Strukturen, Callback-Verträge und Lebensdauer im Header und in der Subsystem-dokumentation prüfen.

02 Kontext beachten

GPIO-Callbacks, IRQ-Pfade und mögliche Sleep-Kontexte folgen unterschiedlichen Regeln.

Subsystem-APIs erklären, welche Aufgaben der konkrete Controller-Treiber übernehmen muss.

Device-Tree “compatible” Strings führen zum Treiber

DTS → BINDING → TREIBER

```
/* Hardwarebeschreibung: vereinfachtes Beispiel */  
gpio@20000 {  
    compatible = "snps,dw-apb-gpio";  
    reg = <0x20000 0x1000>;  
};
```

```
$ git grep -n -F 'snps,dw-apb-gpio' -- \  
    arch/arm64/boot/dts/ drivers/gpio/ \  
    Documentation/devicetree/bindings/gpio/
```

01 Binding lesen

Pflichtfelder,
Child-Nodes und
GPIO-Eigenschaften der
realen
Hardwarebeschreibung
prüfen.

02 Match finden

Der gleiche
compatible-String
erscheint in der
Match-Tabelle des
zugehörigen Treibers.

DTS-Ausschnitt ist vereinfacht: Das vollständige Binding enthält weitere erforderliche Eigenschaften.

Match-Tabelle und probe markieren den Einstieg

AUSZUG · GPIO-DWAPB.C · GEKÜRZT

```
static const struct of_device_id dwapb_of_match[] = {
    { .compatible = "snps,dw-apb-gpio", ... },
    { /* Sentinel */ }
};

MODULE_DEVICE_TABLE(of, dwapb_of_match);

static int dwapb_gpio_probe(
    struct platform_device *pdev)
{ /* Ressourcen holen und Ports registrieren */ }
```

01 Match-Daten

Hardwarekennung und zugehörige Controller-Variante werden über die Tabelle verknüpft.

02 Initialisierung

probe liest Ressourcen, ordnet Register zu und verarbeitet die vorhandenen GPIO-Ports.

Der Ausschnitt ist didaktisch gekürzt; die vollständige Implementierung bleibt die Referenz.

Den tatsächlichen Initialisierungspfad verfolgen

`dwapb_gpio_probe()`

Plattformgerät, Reset, Registerbereich und Clocks prüfen

`dwapb_gpio_add_port()`

Einen Controller-Port aufbauen und konfigurieren

`gpio_generic_chip_init()`

Generische GPIO-Chip-Struktur initialisieren

`devm_gpiochip_add_data()`

Controller bei gpiolib ressourcenverwaltet registrieren

Fehlerpfade, IRQ-Konfiguration und optionale Clocks bewusst mitlesen – nicht nur den Happy Path.

Kconfig und Kbuild entscheiden über den Treiber

KONFIGURATION → OBJEKTDATTEI

```
# drivers/gpio/Kconfig
config GPIO_DWAPB
    tristate "Synopsys DesignWare APB GPIO driver"
    select GPIO_GENERIC
    select GPIOLIB_IRQCHIP

# drivers/gpio/Makefile
obj-$(CONFIG_GPIO_DWAPB) += gpio-dwapb.o

$ grep '^CONFIG_GPIO_DWAPB=' "$build_dir/.config"
```

01 y oder m

Eingebauter Treiber beziehungsweise ladbares Modul – sofern die Konfiguration dies erlaubt.

02 Abhängigkeiten

GPIO_GENERIC und GPIOLIB_IRQCHIP werden vom konkreten Symbol selektiert.

Vorhandener Quellcode bedeutet nicht, dass der Treiber im laufenden Zielkernel aktiviert ist.

Git-Historie erklärt Änderungen und Codeabsicht

NUR LOKALE ANALYSE · OHNE EXTERNE AKTION

```
$ git log --oneline -- \
    drivers/gpio/gpio-dwapb.c
```

```
$ git blame -L 200,260 -- \
    drivers/gpio/gpio-dwapb.c
```

```
$ git log -L :dwapb_gpio_probe:\
    drivers/gpio/gpio-dwapb.c
```

```
$ git grep -n 'GPIO SUBSYSTEM' -- MAINTAINERS
```

01 Commit-Kontext

git log zeigt Änderungen; Commit-Texte und Diffs erklären ihre Motivation.

02 Zeilen und Funktion

git blame ordnet Zeilen zu; git log -L verfolgt die Historie einer Funktion.

Zeilennummern sind exemplarisch und ändern sich mit Version, Branch und lokalem Quellstand.

Tags und cscope beschleunigen die Symbolnavigation

VORAUSSETZUNG · PASSENDE TOOLS INSTALLIERT

```
$ make tags
$ make cscope

$ make ALLSOURCE_ARCHS=arm64 tags
$ make IGNORE_DIRS="tools Documentation" \
    cscope

# Editor: Definition suchen, Referenzen folgen
# Indizes bei Änderungen neu erzeugen
```

01 Symbolindex

ctags ermöglicht schnelle Sprünge zu Funktionen, Typen und weiteren Definitionen.

02 Suchdatenbank

cscope hilft bei Funktionen, Referenzen und Include-Bezügen in großen Quellbäumen.

ALLSOURCE_ARCHS und IGNORE_DIRS steuern Umfang und Kosten der erzeugten Navigationsindizes.

clangd benötigt reale Compiler- und Include-Pfade

IDE-NAVIGATION · BUILD-OUTPUT

```
# Nach einem geeigneten Kernel-Build:
$ python3 scripts/clang-tools/\
gen_compile_commands.py \
    -d "$build_dir" \
    -o "$build_dir/compile_commands.json"

# clangd / Editor mit dem Output-Tree verbinden:
--compile-commands-dir=/tmp/kernel-build
```

01 Echte Build-Daten

Das Kernel-Skript gewinnt Compilerbefehle aus vorhandenen .cmd-Dateien.

02 Ziel richtig wählen

Architektur, .config, generierte Header und Include-Pfade müssen zum Build passen.

Ohne vorherigen Build fehlen relevante .cmd-Dateien – dann bleibt die Datenbank unvollständig.

Praxisaufgabe: Einen GPIO-Treiber vollständig finden

01 Hardware und Binding identifizieren

Nach snps,dw-apb-gpio in DTS-Dateien und der GPIO-Binding-Dokumentation suchen.

02 Implementierung und Initialisierung lesen

dwapb_of_match, dwapb_gpio_probe und dwapb_gpio_add_port im Treiber verfolgen.

03 Konfiguration und Build kontrollieren

GPIO_DWAPB in Kconfig, drivers/gpio/Makefile und der aktiven .config überprüfen.

04 Subsystem und User-Space zuordnen

devm_gpiochip_add_data, struct gpio_chip und die GPIO-Character-Device-UAPI einordnen.

Ergebnis: Ein belastbarer Analysepfad von Hardwarekennung über Treiber und Build bis zur UAPI.